

AWS Services – Self-Learning & Hands-on Implementation

1. Introduction

As part of the continuous learning and hands-on implementation initiative, the following AWS services were studied and implemented:

1. **Amazon SQS – Simple Queue Service**
2. **Amazon MQ**
3. **AWS Secrets Manager**

The objective of this activity is to understand the concepts, use cases, architecture, configuration, and practical implementation of these AWS services.

2. Amazon SQS – Simple Queue Service

2.1 Objective

To understand and implement Amazon SQS for reliable and asynchronous communication between application components.

2.2 Concept

Amazon SQS is a fully managed message queuing service that enables applications to communicate asynchronously.

Instead of the producer directly communicating with the consumer, the producer sends a message to an SQS queue. The consumer retrieves and processes the message from the queue.

Key Components

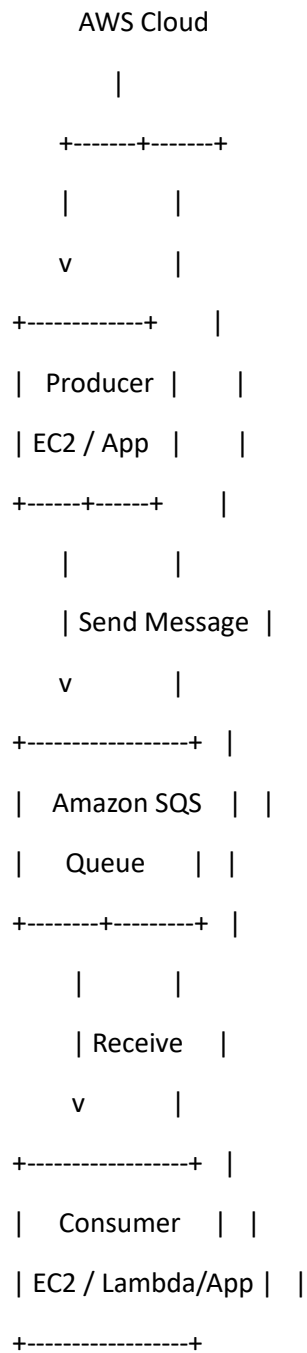
- **Producer** – Sends messages to the queue.
- **Queue** – Temporarily stores messages.
- **Consumer** – Retrieves and processes messages.
- **Visibility Timeout** – Prevents other consumers from processing a message while it is being processed.
- **Dead-Letter Queue (DLQ)** – Stores messages that cannot be successfully processed after multiple attempts.

2.3 Use Cases

- Decoupling microservices
- Background job processing
- Order processing
- Asynchronous application communication

- Handling traffic spikes
- Event-driven applications

2.4 Architecture



2.5 Implementation

Step 1 – Open Amazon SQS

1. Log in to the AWS Management Console.
2. Select the required AWS Region.

3. Search for **SQS**.
4. Open **Amazon Simple Queue Service**.

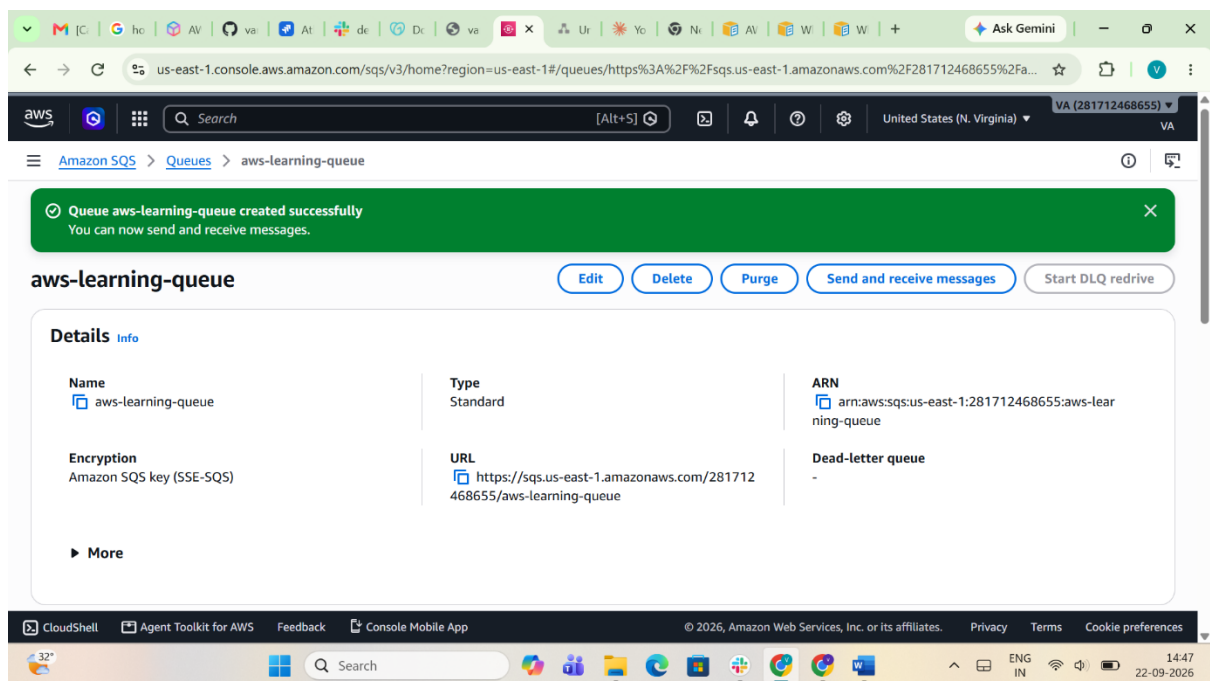
Step 2 – Create Queue

1. Click **Create queue**.
2. Select **Standard** queue.
3. Enter the queue name:

aws-learning-queue

4. Keep the default configuration for the basic implementation.
5. Click **Create queue**.

Screenshot 1: SQS Queue Creation



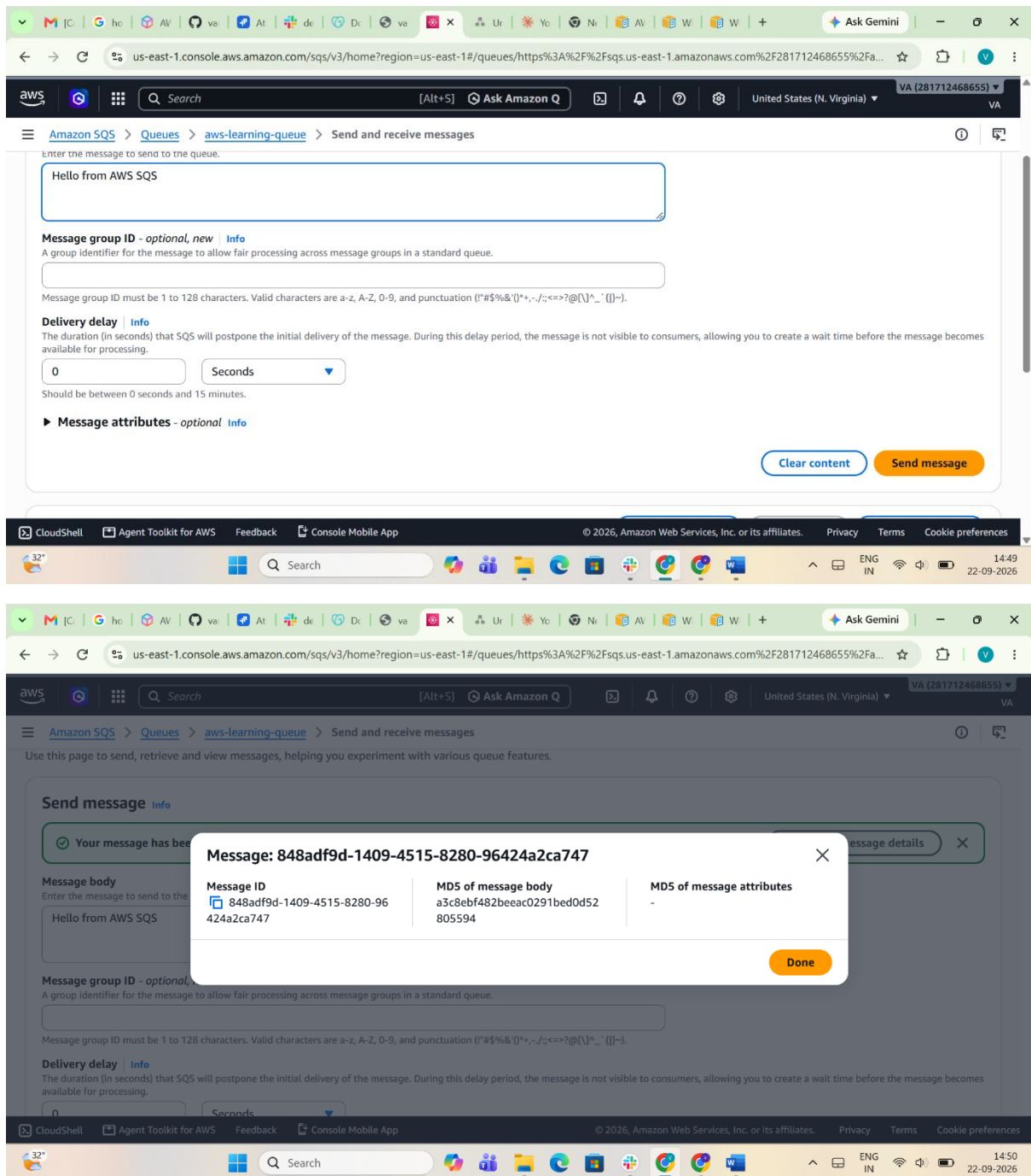
Step 3 – Send Message

1. Open the newly created queue.
2. Click **Send and receive messages**.
3. Enter the following message:

Hello from AWS SQS

4. Click **Send message**.

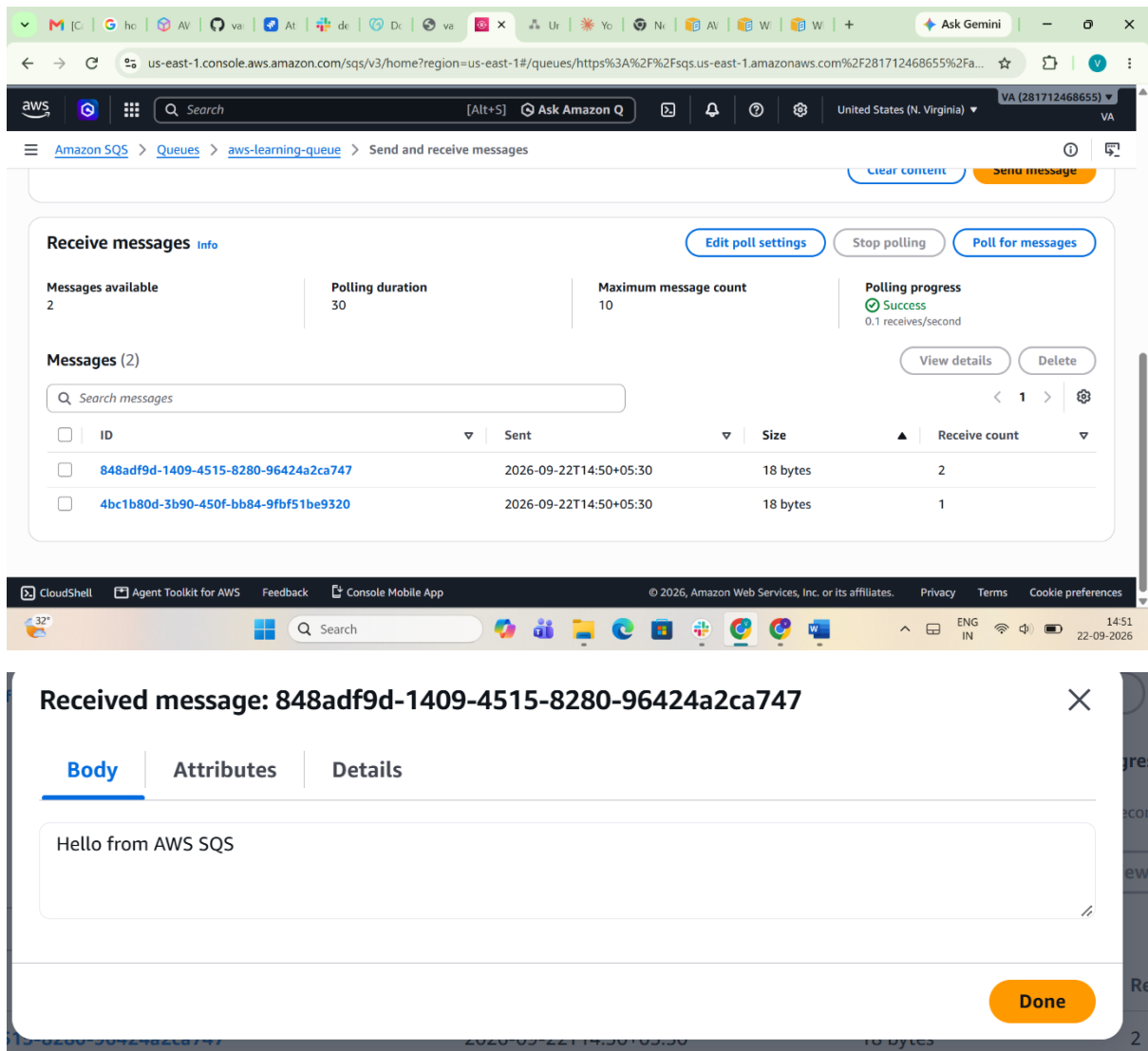
Screenshot 2: SQS Message Sent



Step 4 – Receive Message

1. Click **Poll for messages**.
2. Verify that the message is displayed.
3. Open the message and verify its content.

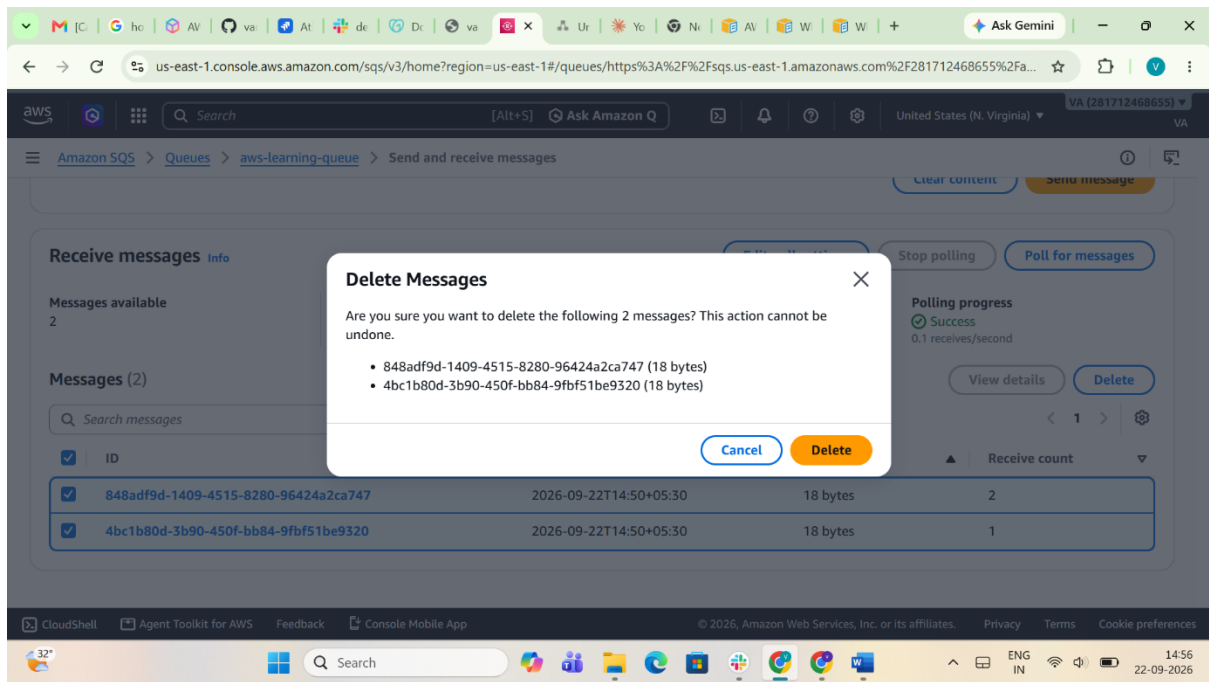
Screenshot 3: SQS Message Received



Step 5 – Delete Message

1. Select the processed message.
2. Click **Delete**.
3. Poll for messages again.
4. Verify that the message has been removed.

Screenshot 4: SQS Message Deleted



2.6 Observation

- SQS successfully stored messages sent by the producer.
- Messages could be retrieved by the consumer.
- The consumer could delete the message after successful processing.
- SQS provides asynchronous communication between application components.
- The queue helps decouple the producer and consumer.

2.7 Conclusion

Amazon SQS was successfully implemented to demonstrate message creation, retrieval, and deletion. The implementation shows how SQS can be used to build loosely coupled and scalable applications.

3. Amazon MQ

3.1 Objective

To understand and implement Amazon MQ as a managed message broker for applications that use standard messaging protocols.

3.2 Concept

Amazon MQ is a managed message broker service provided by AWS.

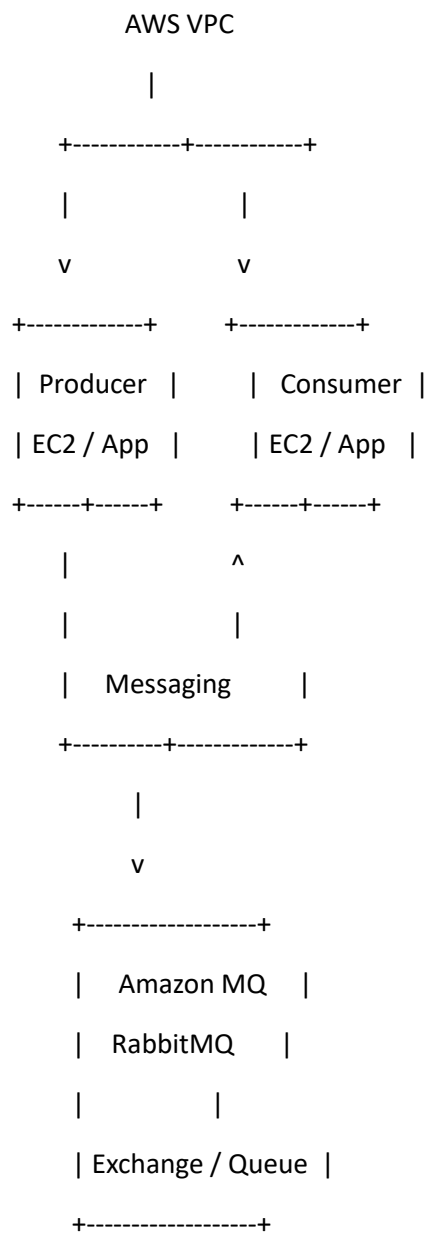
It supports popular messaging engines such as **RabbitMQ** and **ActiveMQ**.

Unlike SQS, which is an AWS-native queue service, Amazon MQ is designed for applications that require traditional message brokers and standard messaging protocols.

3.3 Use Cases

- Migrating existing messaging applications to AWS
- Applications using RabbitMQ or ActiveMQ
- Enterprise messaging
- Application integration
- Message-based communication between distributed applications
- Applications requiring standard messaging protocols

3.4 Architecture



3.5 Implementation

Step 1 – Open Amazon MQ

1. Log in to AWS Console.
2. Select the required Region.
3. Search for **Amazon MQ**.
4. Open Amazon MQ.
5. Click **Create broker**.

Step 2 – Select Broker Engine

1. Select **RabbitMQ**.
2. Select an available RabbitMQ version.
3. Select an appropriate deployment/broker instance option for the lab.

Step 3 – Configure Broker

Enter a broker name:

aws-learning-rabbitmq

Create administrator credentials:

Username: admin

Password: <lab-password>

Do not include the actual password in the implementation document.

Step 4 – Configure Networking

Configure:

- VPC
- Subnet
- Security group
- Network accessibility

For a learning environment, use your existing learning VPC if appropriate.

Step 5 – Configure Security Group

Allow the required RabbitMQ ports from the required source.

For example:

Port	Purpose
------	---------

5672	AMQP messaging
------	----------------

Port Purpose

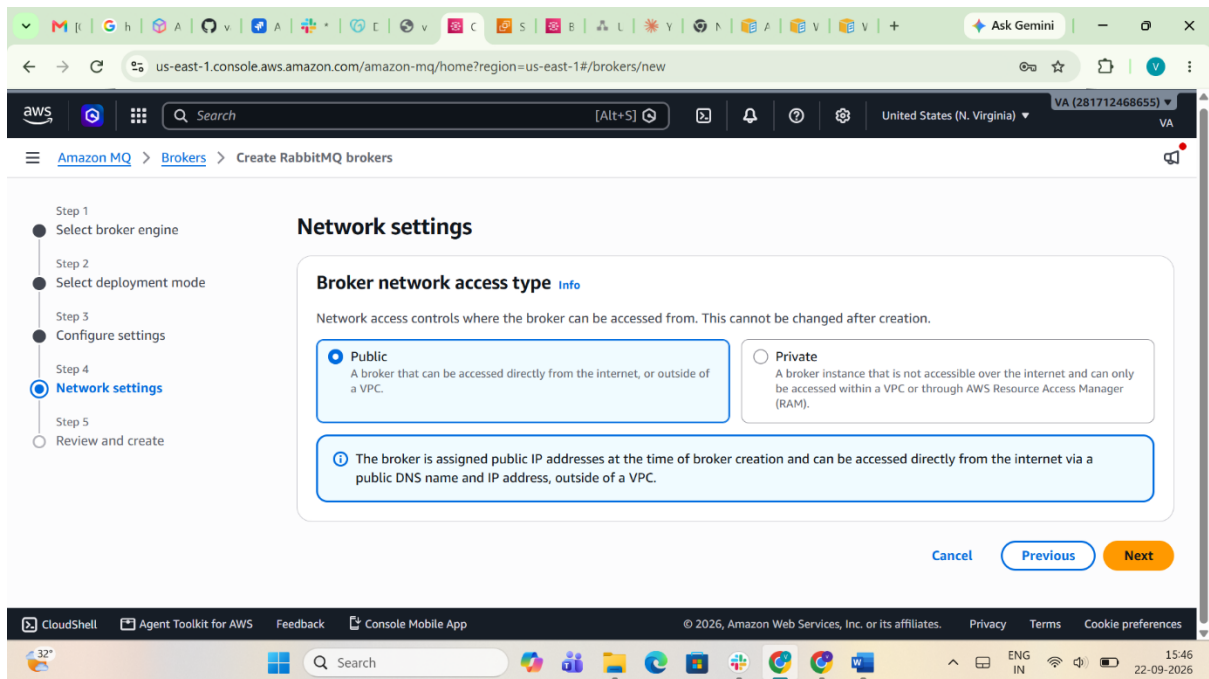
15672 RabbitMQ Management Console

Restrict the source to the required EC2/security group rather than opening access to everyone.

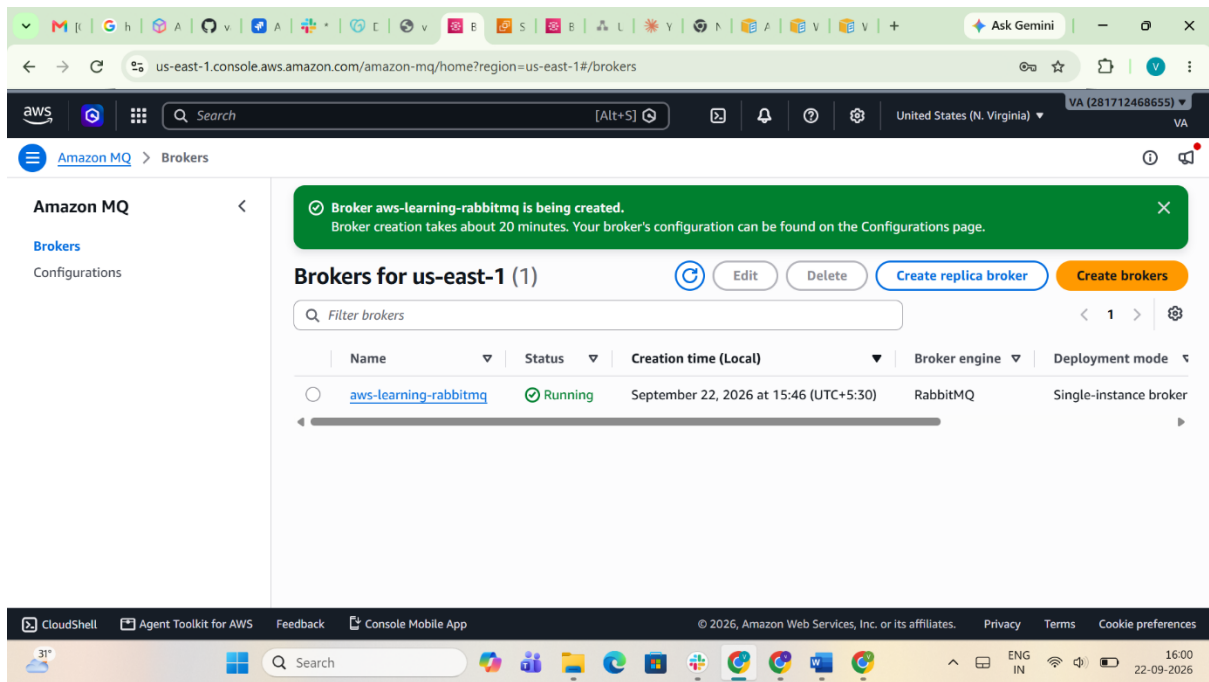
Step 6 – Create Broker

1. Review the configuration.
2. Click **Create broker**.
3. Wait for the broker to become available/running.

Screenshot 1: Amazon MQ Broker Configuration



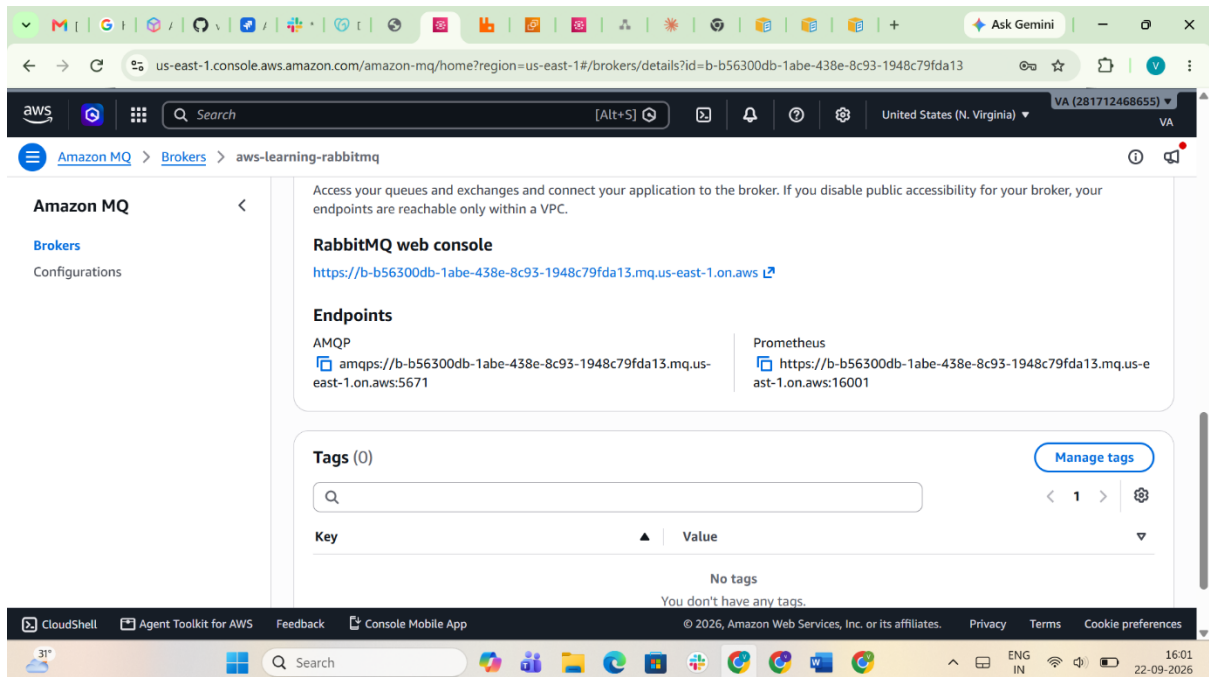
Screenshot 2: Amazon MQ Broker Successfully Created

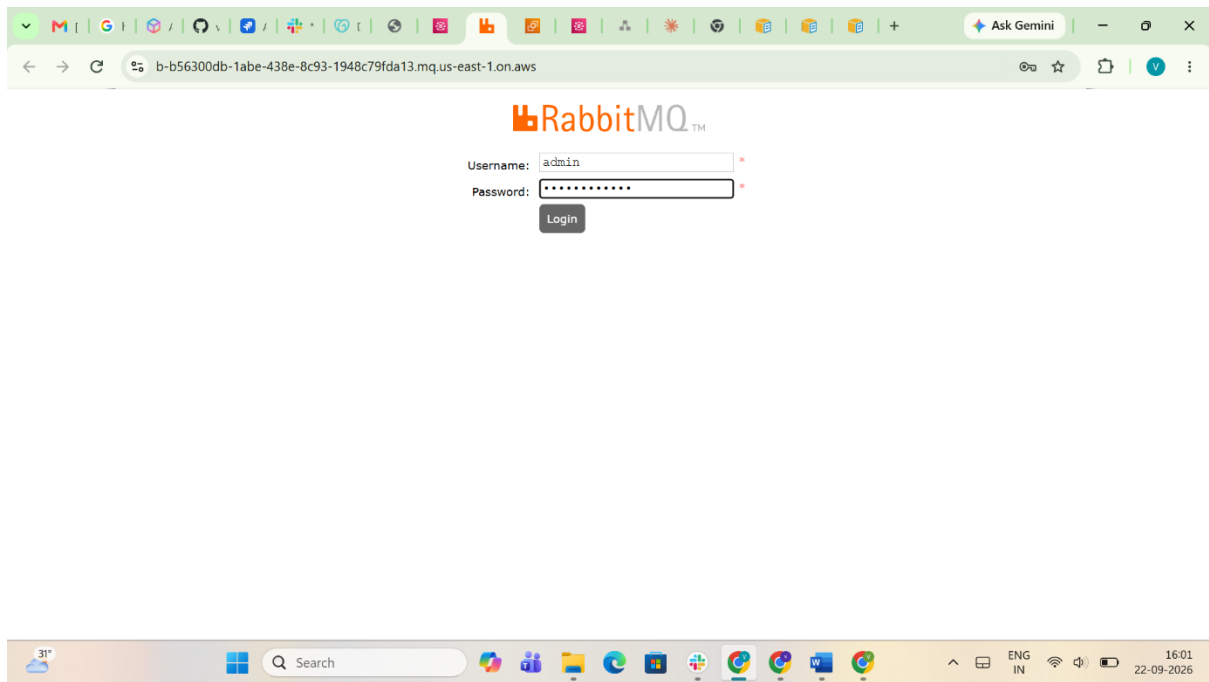


Step 7 – Open RabbitMQ Management Console

1. Open the Amazon MQ broker details.
2. Locate the RabbitMQ management endpoint.
3. Open the endpoint.
4. Log in using the administrator credentials.

Screenshot 3: RabbitMQ Management Console





Step 8 – Create Queue

In the RabbitMQ Management Console:

1. Open **Queues and Streams**.
2. Click **Add a new queue**.
3. Enter:

aws-learning-queue

4. Keep the default settings.
5. Click **Add Queue**.

Screenshot 4: RabbitMQ Queue

RabbitMQ Management Console Overview

Refreshed 2026-09-22 16:02:02 | Refresh every 5 seconds | Virtual host: All | Cluster: aws-learning-rabbitmq | User: admin | Log out

Overview | Connections | Channels | Exchanges | Queues and Streams | Admin

Message rates: last minute ?
Currently idle
Global counts ?

Connections: 0 | Channels: 0 | Exchanges: 8 | Queues: 0 | Consumers: 0

Name	File descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats
rabbit@localhost	45 500000 available	477 1048576 available	145 MiB 4.3 GiB high watermark	187 GiB 10.0 GiB low watermark	7m 52s	2	basic 8 rss	This node All nodes

Churn statistics
Ports and contexts
Export definitions
Import definitions

HTTP API | Documentation | Tutorials | New releases | Commercial edition | Commercial support | Discussions | Discord | Plugins | GitHub

RabbitMQ Management Console - Queue: aws-learning-queue

Refreshed 2026-09-22 16:03:17 | Refresh every 5 seconds | Virtual host: All | Cluster: aws-learning-rabbitmq | User: admin | Log out

Overview | Connections | Channels | Exchanges | Queues and Streams | Admin

Queued messages: last minute ?

Ready: 0
Unacked: 0
Total: 0

Message rates: last minute ?
Currently idle

Details

Features	State
arguments: x-queue-type: quorum	running
durable: true	Consumers: 0
delayed retry: not enabled	Publishers: 0
Policy	Open files: rabbit@localhost: 0
Operator policy	Messages by priority
Effective policy definition	Delivery limit: 20

Step 9 – Verify

Verify:

- Broker is running.
- RabbitMQ Management Console is accessible.
- Queue has been created.
- Messaging components are available.

3.6 Observation

- Amazon MQ provides a managed message broker.
- RabbitMQ provides exchanges, queues, bindings, and standard messaging protocols.
- Network and security-group configuration are important for broker connectivity.
- Amazon MQ is suitable for applications that already depend on traditional message brokers.

3.7 Conclusion

Amazon MQ was successfully configured with RabbitMQ. The broker and queue were created and verified through the RabbitMQ Management Console.

4. AWS Secrets Manager

4.1 Objective

To understand and implement AWS Secrets Manager for securely storing and retrieving sensitive application credentials.

4.2 Concept

AWS Secrets Manager is a service used to securely store sensitive information such as:

- Database usernames/passwords
- API keys
- Application credentials
- Access tokens
- Other confidential configuration data

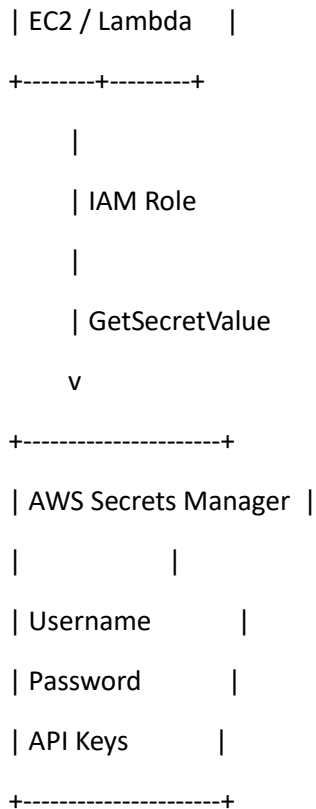
Instead of storing credentials directly inside application code, applications can retrieve them securely from Secrets Manager.

4.3 Use Cases

- Storing database credentials
- Storing API keys
- Application authentication
- Secure configuration management
- Credential rotation
- Centralized secret management

4.4 Architecture

```
+-----+
| Application |
```



4.5 Implementation

Step 1 – Open Secrets Manager

1. Log in to AWS Console.
2. Select the required AWS Region.
3. Search for **Secrets Manager**.
4. Open the service.

Step 2 – Create Secret

1. Click **Store a new secret**.
2. Select:

Other type of secret

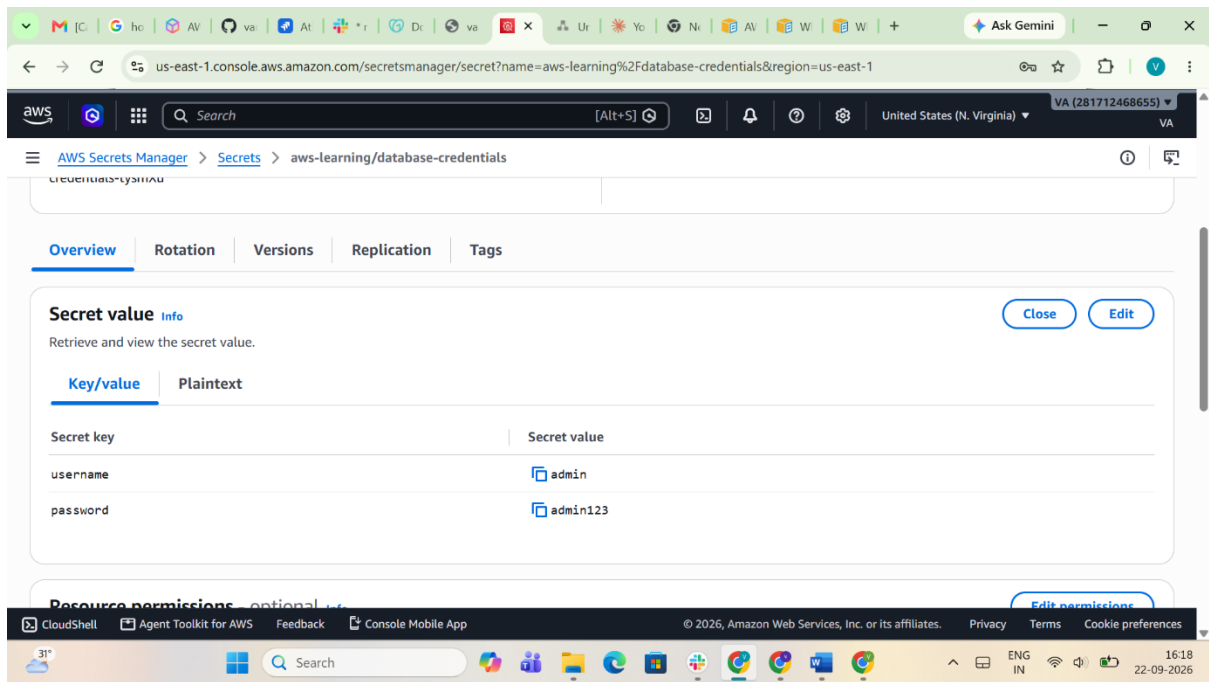
3. Add sample key/value pairs:

username = admin

password = MyLabPassword123

Use a lab password and do not expose the actual password in the document.

Screenshot 1: Secrets Manager Secret Configuration



Step 3 – Configure Secret Name

Enter:

aws-learning/database-credentials

Add a description:

Database credentials for AWS learning application

Click **Next**.

Step 4 – Configure Rotation

For the basic implementation:

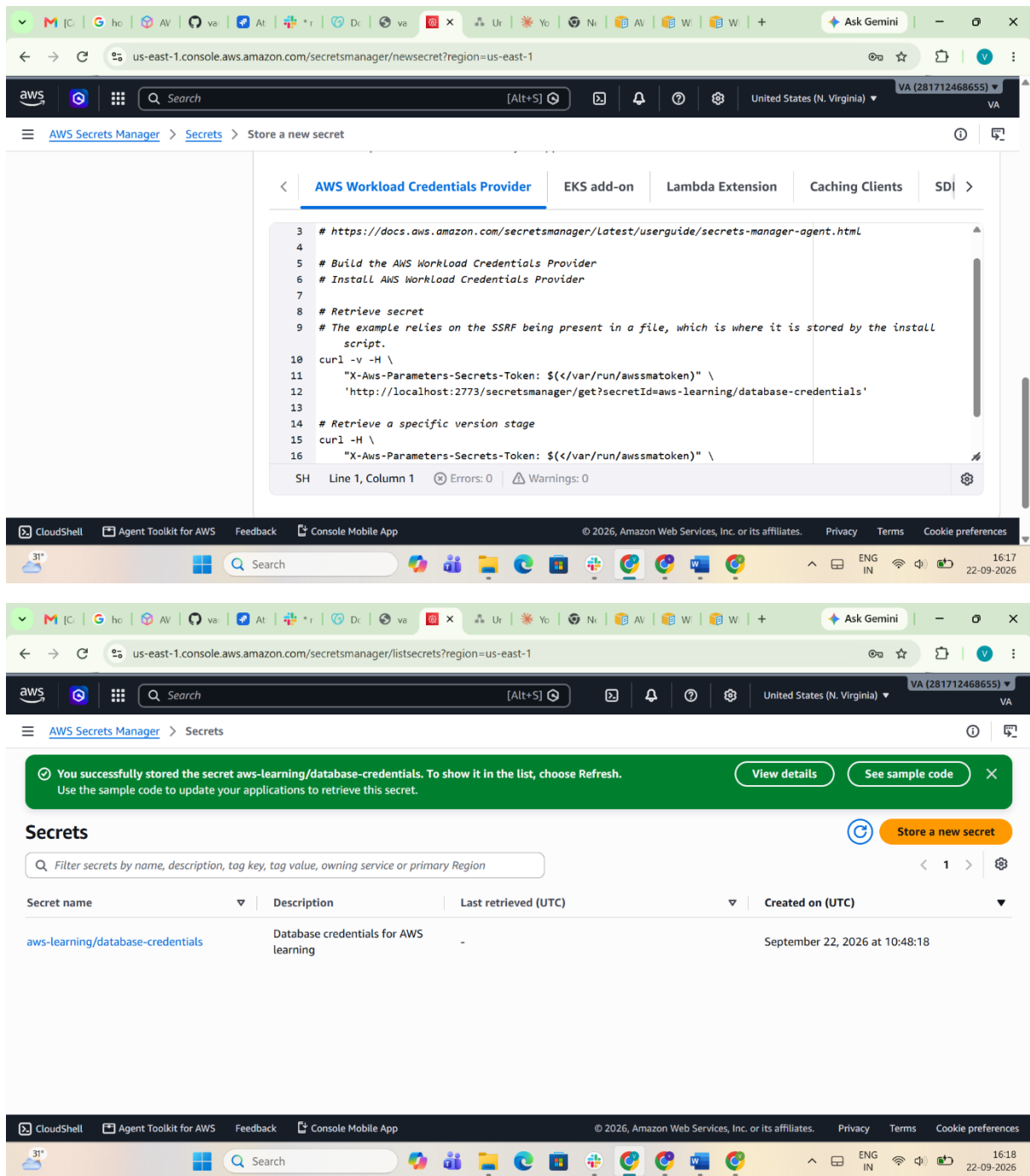
1. Keep automatic rotation disabled.
2. Continue to the next step.

Rotation can be configured separately for supported services.

Step 5 – Store Secret

1. Review the configuration.
2. Click **Store**.
3. Verify that the secret has been successfully created.

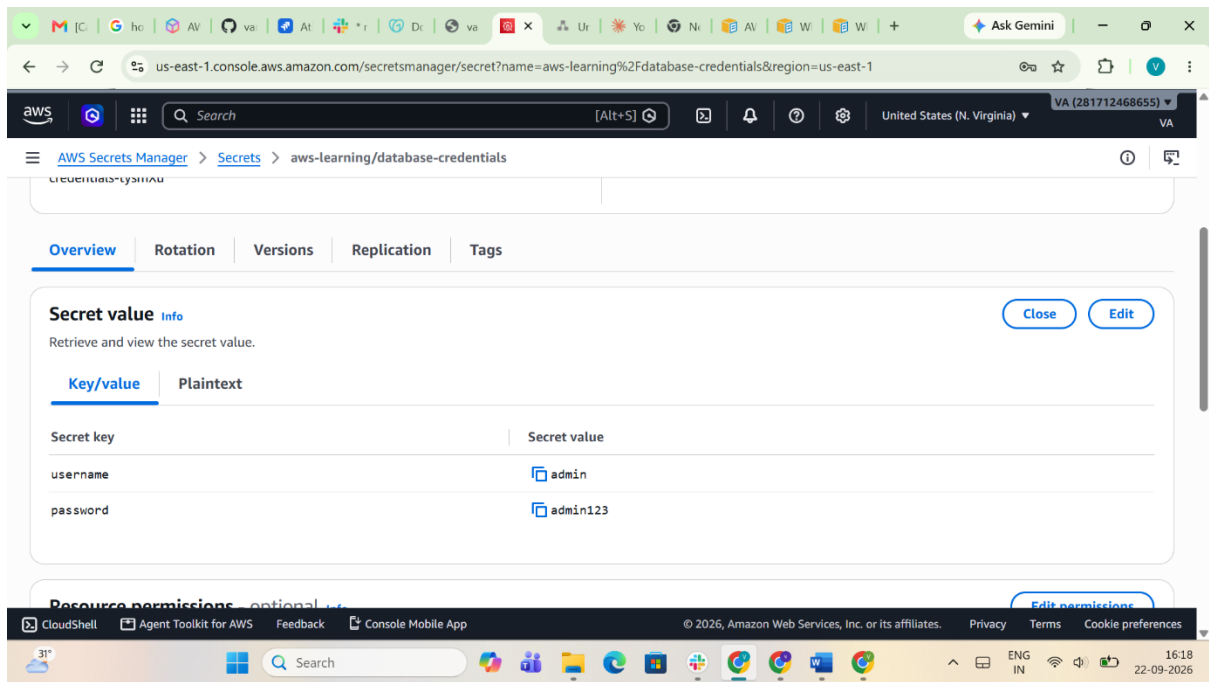
Screenshot 2: Secret Successfully Created



Step 6 – Retrieve Secret

1. Open the created secret.
2. Click **Retrieve secret value**.
3. Verify the stored key/value pairs.

Screenshot 3: Retrieve Secret Value



Mask the password before adding the screenshot to your document.

Step 7 – Configure IAM Permission

The application needs permission to retrieve the secret.

Create or use an IAM role for the application and provide the required permission:

secretsmanager:GetSecretValue

Where possible, restrict the permission to the specific secret ARN.

Architecture:

EC2 / Lambda

|

| IAM Role

|

| GetSecretValue

v

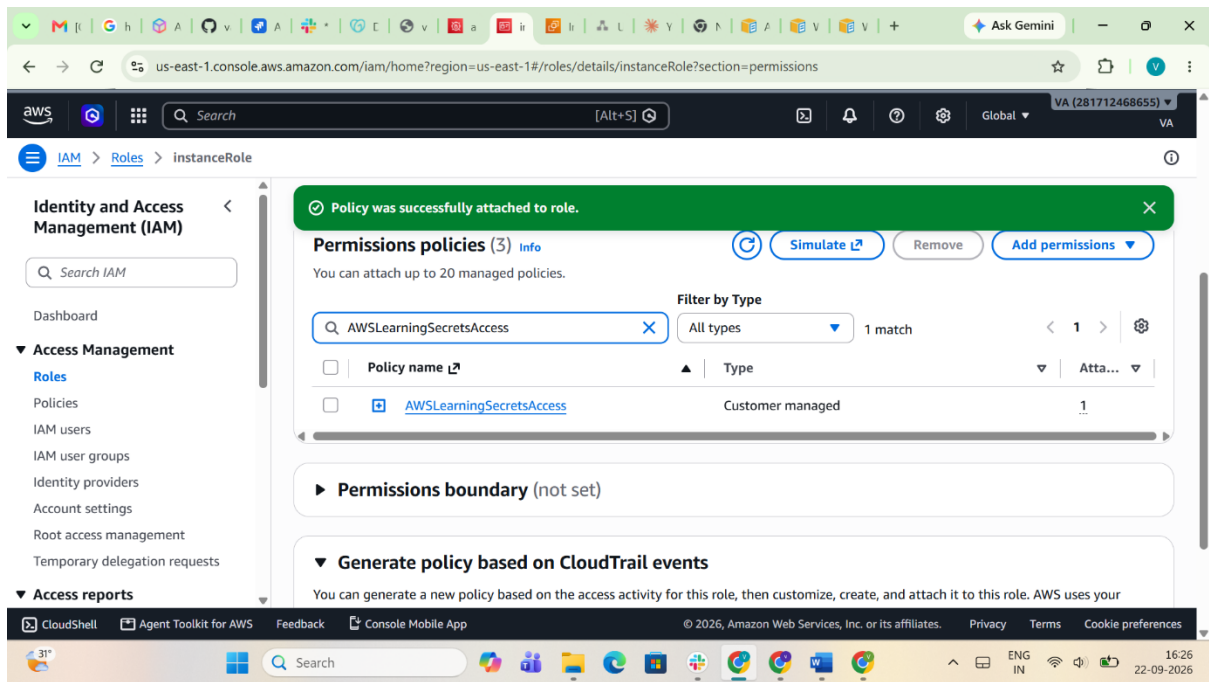
Secrets Manager

|

v

Secret Value

Screenshot 4: IAM Permission

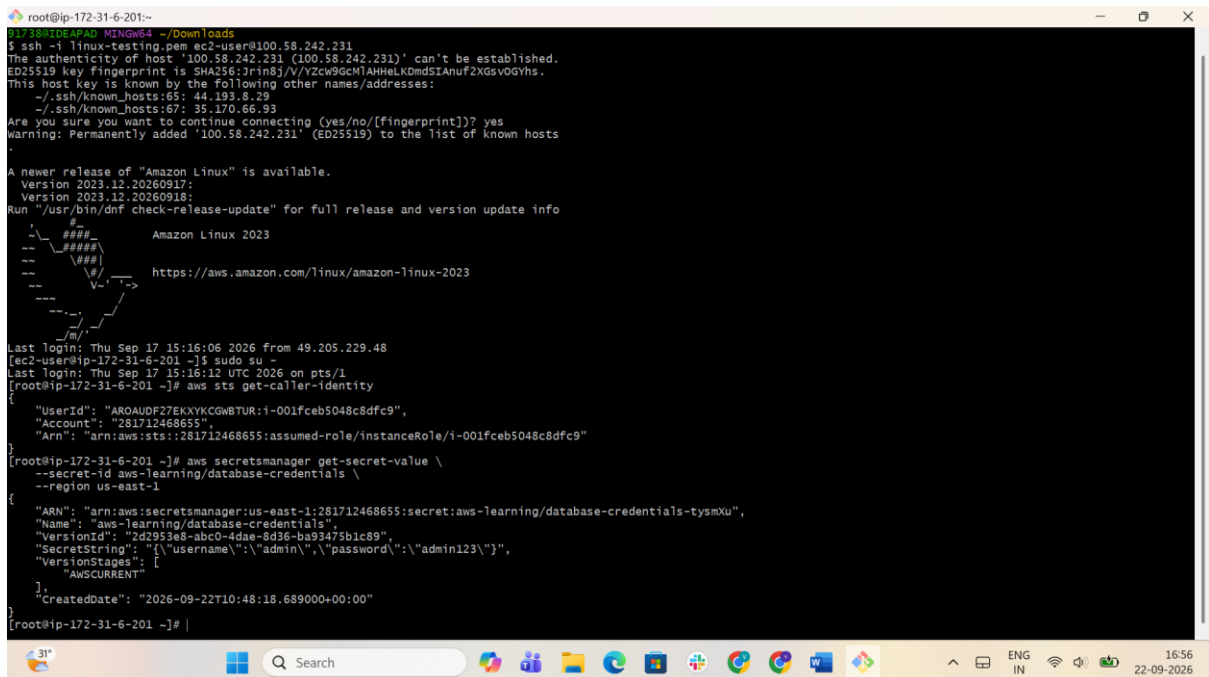


Step 8 – Test Secret Retrieval

You can validate access using the AWS CLI:

```
aws secretsmanager get-secret-value \
    --secret-id aws-learning/database-credentials \
    --region <your-region>
```

If the IAM permissions are correct, the secret value will be returned.



Step 9 – Audit Secret Access

AWS CloudTrail can be used to monitor Secrets Manager API activity.

Examples:

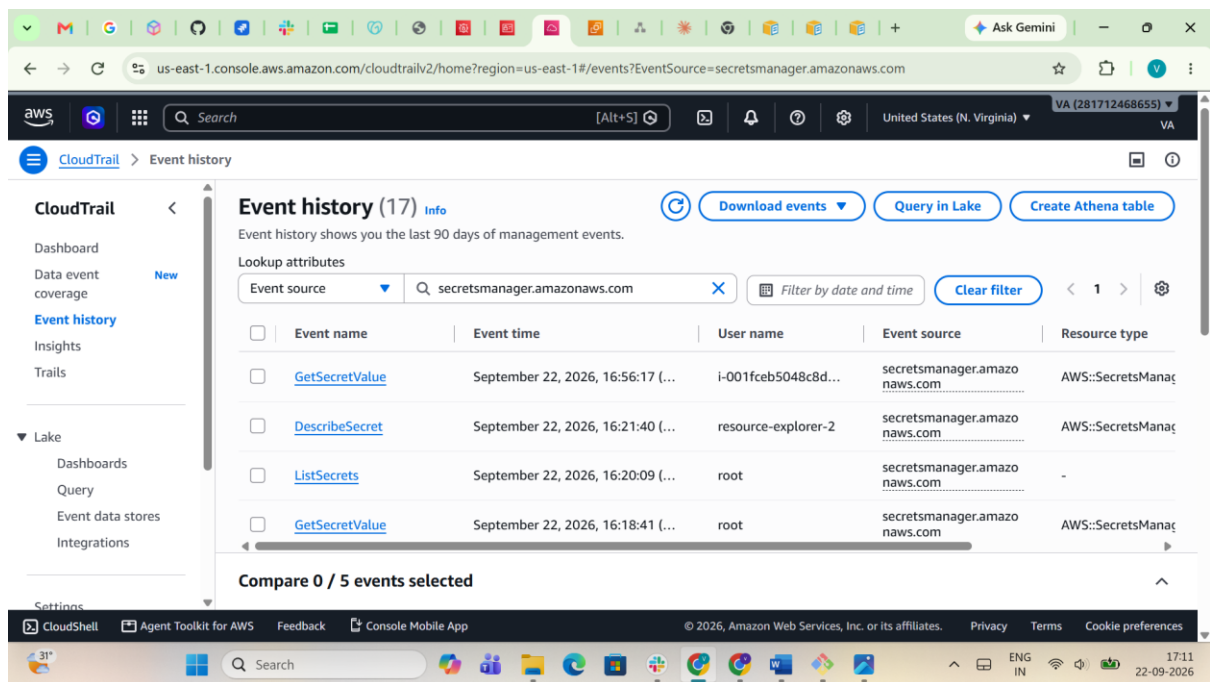
CreateSecret

GetSecretValue

UpdateSecret

DeleteSecret

Screenshot 5: CloudTrail Secret Activity



4.6 Observation

- Secrets Manager securely stores sensitive information.
- Applications do not need to hardcode credentials.
- IAM controls access to secrets.
- Secret access can be audited using CloudTrail.
- Secrets Manager supports secret rotation for supported use cases.

4.7 Conclusion

AWS Secrets Manager was successfully implemented to create, securely store, and retrieve application credentials. IAM permissions were used to control access to the secret.

5. SQS vs Amazon MQ

Feature	Amazon SQS	Amazon MQ
Type	Message queue	Message broker
AWS managed	Yes	Yes
RabbitMQ/ActiveMQ	No	Yes
Server management	No	AWS manages broker infrastructure
Messaging model	Queue-based	Broker-based
AWS-native applications	Commonly used	Can be used
Existing broker applications	Requires migration/adaptation	Suitable
Infrastructure configuration	Minimal	More networking/configuration

6. Overall Observations

During the hands-on implementation:

1. **SQS** was used to create a queue and demonstrate sending, receiving, and deleting messages.
2. **Amazon MQ** was used to create a managed RabbitMQ broker and configure a messaging queue.
3. **Secrets Manager** was used to securely store and retrieve sensitive credentials.
4. IAM permissions are important when applications need access to AWS services.
5. CloudTrail can be used to audit API activity for services such as Secrets Manager.

7. Overall Conclusion

The hands-on implementation provided practical understanding of three AWS services used for messaging and secure application configuration. SQS demonstrated asynchronous message queuing, Amazon MQ demonstrated managed traditional message-broker functionality, and Secrets Manager demonstrated secure storage and controlled retrieval of application credentials.